

EGE353

Autonomous Mobile Robotics

Service mobile robots, concepts of path planning, navigation, obstacle avoidance, simultaneous localisation and mapping.

Lab 1: Understanding Ros nodes and Turtlesim

Chan Kit Wai
Tel: 65500559
Chan_kit_wai@nyp.edu.sg



Synopsis

This module aims to equip learners with the knowledge of service robots and their applications. Learners will also study the concepts of path planning, navigation, obstacle avoidance, simultaneous localisation and mapping. They will be able to implement mobile robotics system to carry out tasks autonomously.

Module Learning Outcomes

At the end of the module, learners are able to:

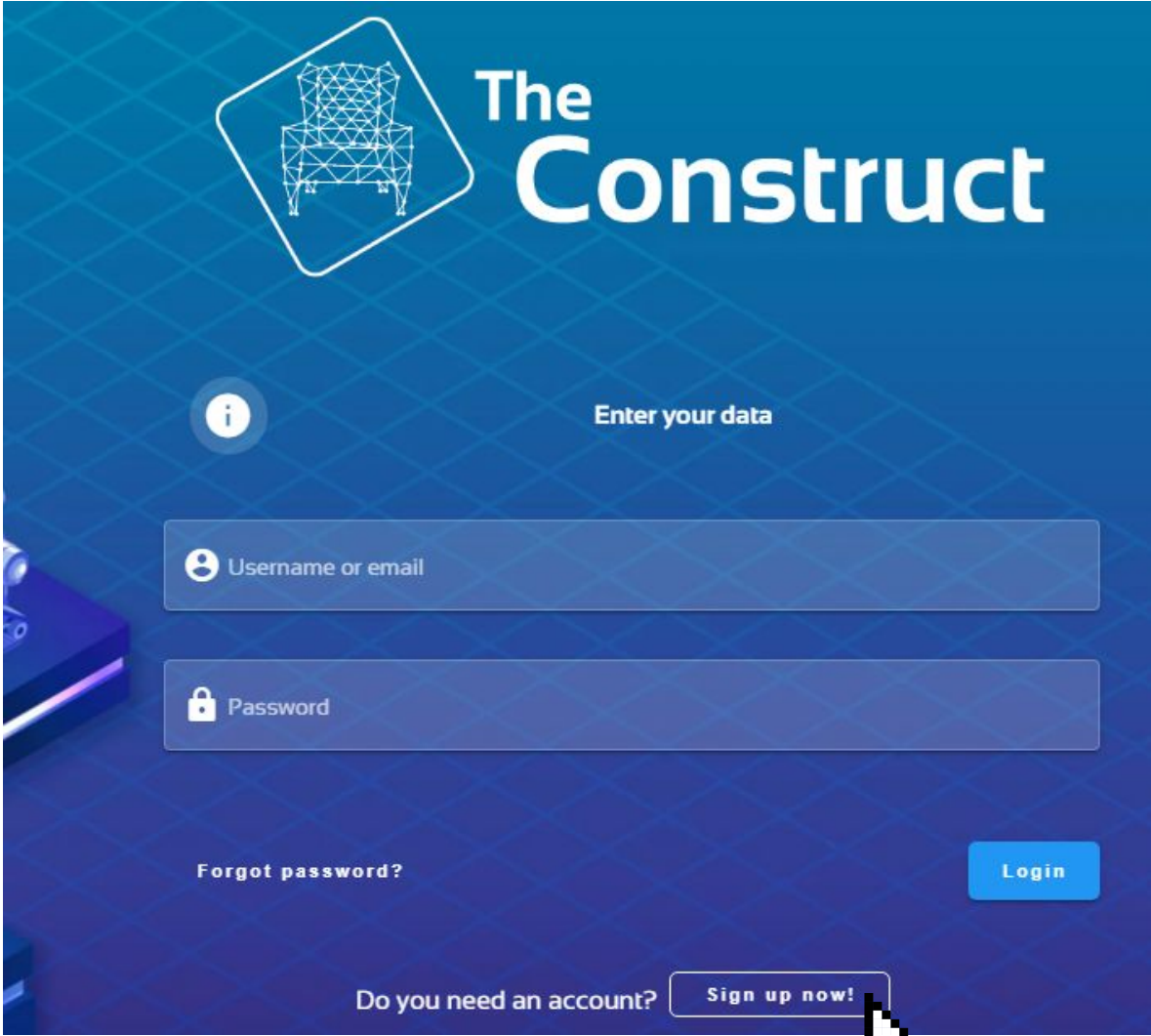
- Explain the features of different sensors and actuators used in autonomous mobile robotics.
- Perform the analysis of laser and vision sensor data using visualization tools.
- Implement mobile robotics system using methods such as mapping, localization, path planning, obstacle avoidance and motion control.

Web-based simulator

<https://www.theconstruct.ai/>

<https://app.theconstruct.ai/signup/>

Make sure you
record your email
and password
somewhere



The Construct

Enter your data

Username or email

Password

Forgot password?

Login

Do you need an account? Sign up now!

A screenshot of the 'The Construct' web-based simulator login and signup page. The page has a dark blue background with a subtle grid pattern. At the top left is a logo consisting of a white wireframe chair inside a rounded square. To the right of the logo is the text 'The Construct' in a large, white, sans-serif font. Below the logo and text is a section titled 'Enter your data' with a small information icon (i) to its left. This section contains two input fields: 'Username or email' and 'Password'. Below these fields are two links: 'Forgot password?' and a blue 'Login' button. At the bottom of the page, there is a text prompt 'Do you need an account?' followed by a button labeled 'Sign up now!'. A white mouse cursor is pointing at the 'Sign up now!' button.

Web-based simulator

The Construct

Type and press enter

Home

Learn

Self-led courses

Instructor-led ☒

Develop

Public Rosjects

My Rosjects

Real Robots

Jobs

ROS Jobs

Your personal ROS newsfeed

Your recent courses [Explore other courses](#)

Course Title	Rating	Reviews	Progress	Duration	Action
ROS2 Basics in 5 Days (Python) NEW!	★★★★☆ 4.4	257 ratings	0% completed	20 hours	Open Course
Developing Web Interfaces for ROS	★★★★★ 4.9	20 ratings	8% completed	120 hours	Open Course
RTAB-Map in ROS IOI	★★★★☆ 4	38 ratings	33% completed	1 hours	Open Course

Your recent rosjects [Explore other rosjects](#)

Web-based simulator

Create new rosject

✓
Project Type

2
ROS Distro

3
Simulation

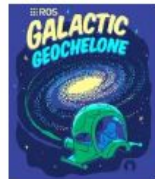
4
Project Data

Which ROS distribution do you want to use?

ROS 1



Noetic

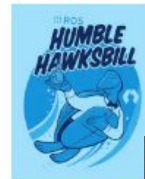


Galactic



Jazzy

ROS 2



Humble



Iron

Web-based simulator

Create new rosject



Project Type



ROS Distro



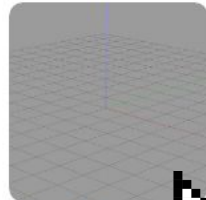
Simulation



Project Data

Which simulation would you like to use?

Select one of the options below



Create from scratch



Compose a simulation

Web-based simulator

Create new rosject



Project Type



ROS Distro



Simulation



Project Data

How about a working title?

You can change it later

Project Name

EGE353

6 / 60

CREATE



Web-based simulator

English

Home → My rosjects → EGE353

 **EGE353**
chankeyzeon PUBLIC

 Delete

 Download

 Share

 Run

Project preview

This rosject does not have a notebook preview



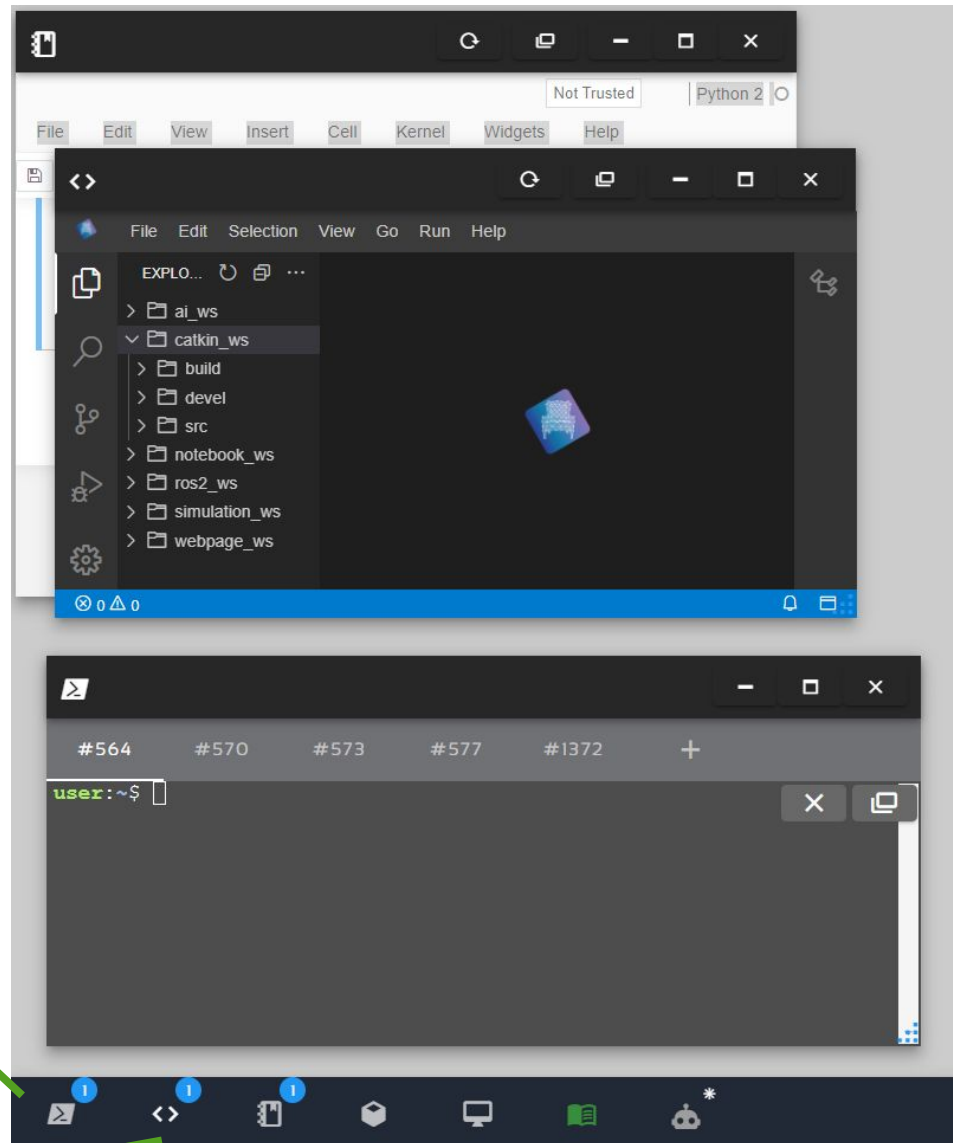
EGE353

No tags added for this rosject

 ROS2 Humble

 0.00 KB

Web-based simulator



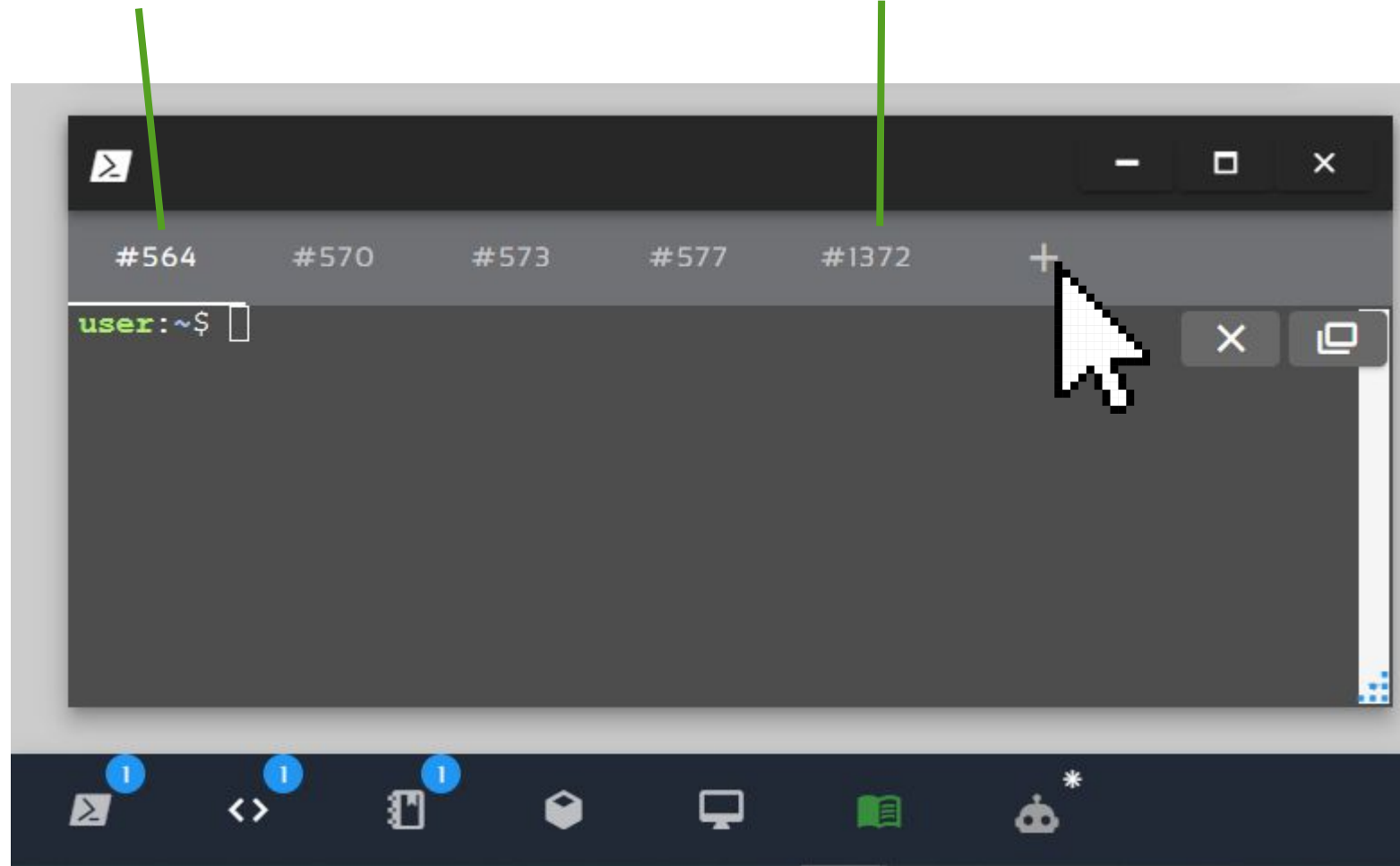
Click to open a new shell /
terminal window

Click to open code editor

Web-based simulator

shell 1 / terminal window 1

shell 5 / terminal window 5

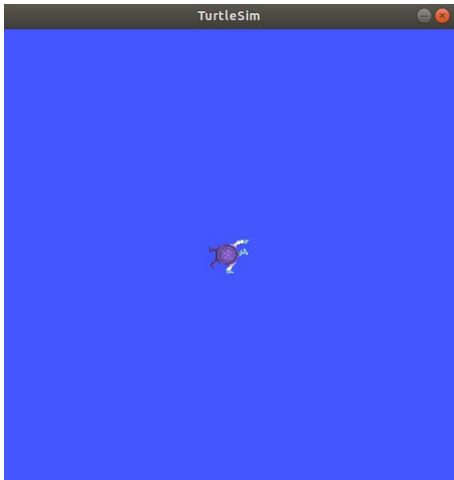


Nodes in ROS2

To start turtlesim, enter the following command in your terminal:

```
source /opt/ros/humble/setup.bash
```

```
ros2 run turtlesim turtlesim_node
```



The simulator window should appear, with a random turtle in the center. In the terminal under the command, you will see messages from the node. You can use `ctr + mouse scroll` to adjust the web browser zoom to 100%.

```
[INFO] [turtlesim]: Starting turtlesim with node name /turtlesim
```

```
[INFO] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
```

Nodes in ROS2

Open a 2nd terminal while turtlesim is still running in the 1st terminal, and enter the following command:

```
source /opt/ros/humble/setup.bash
```

```
ros2 node list
```

The terminal will return the node name /turtlesim as there is only one node running.

Open a 3rd terminal while turtlesim is still running in the 1st terminal, and enter the following command:
Pls ensure that you click on the 3rd terminal before pressing the arrow key

```
ros2 run turtlesim turtle_teleop_key
```

Reading from keyboard

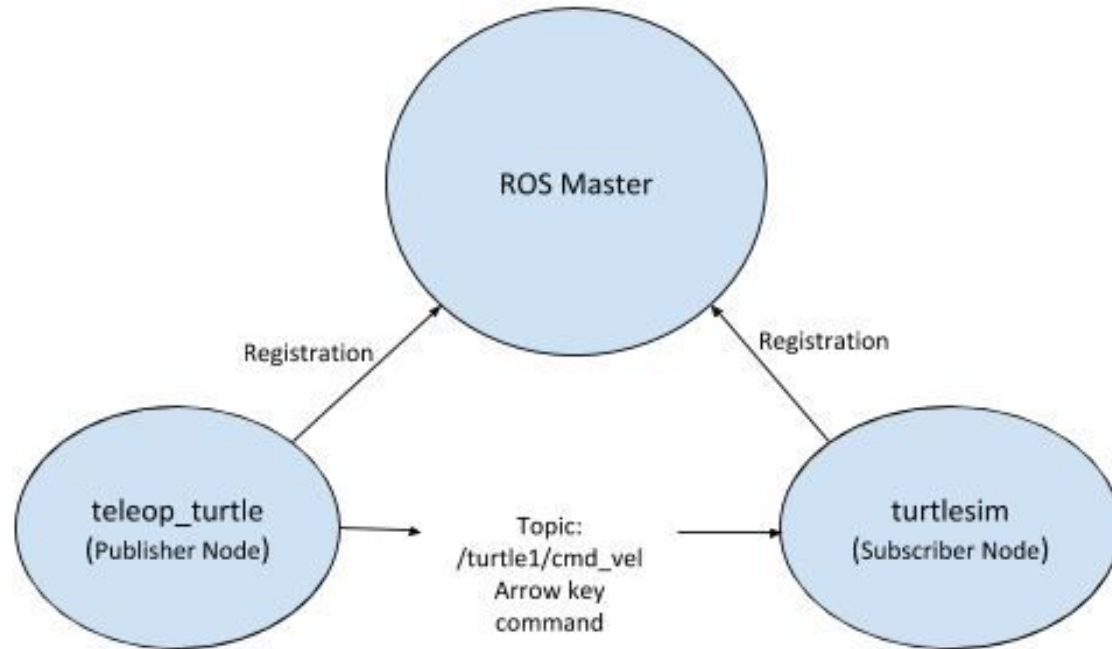
Use arrow keys to move the turtle.

Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rotation.

'Q' to quit.



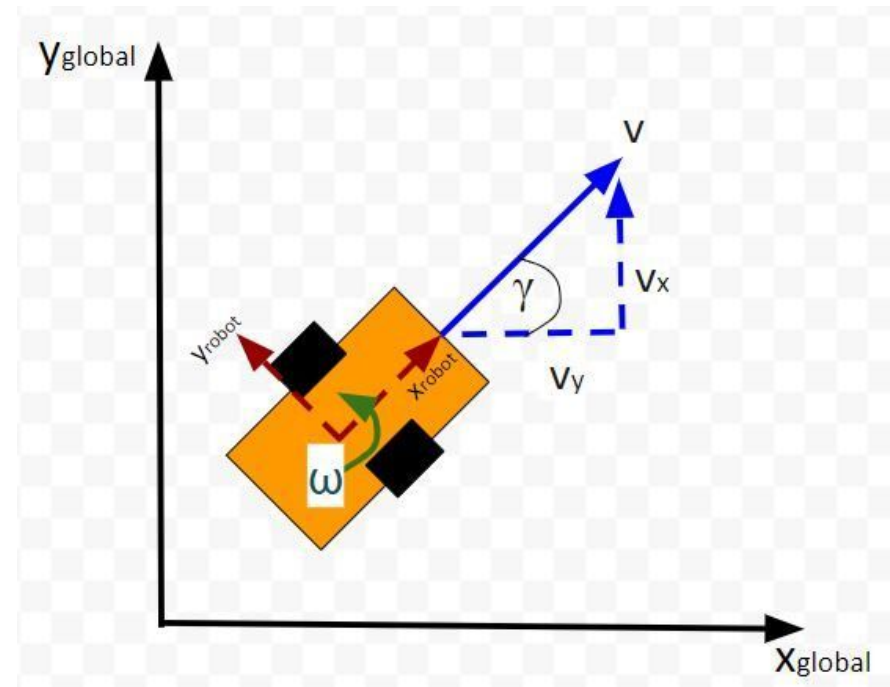
Turtlesim



```
/turtlesim
user:~$ ros2 run turtlesim turtle_teleop_key
Reading from keyboard
-----
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. '
F' to cancel a rotation.
'Q' to quit.
█
```

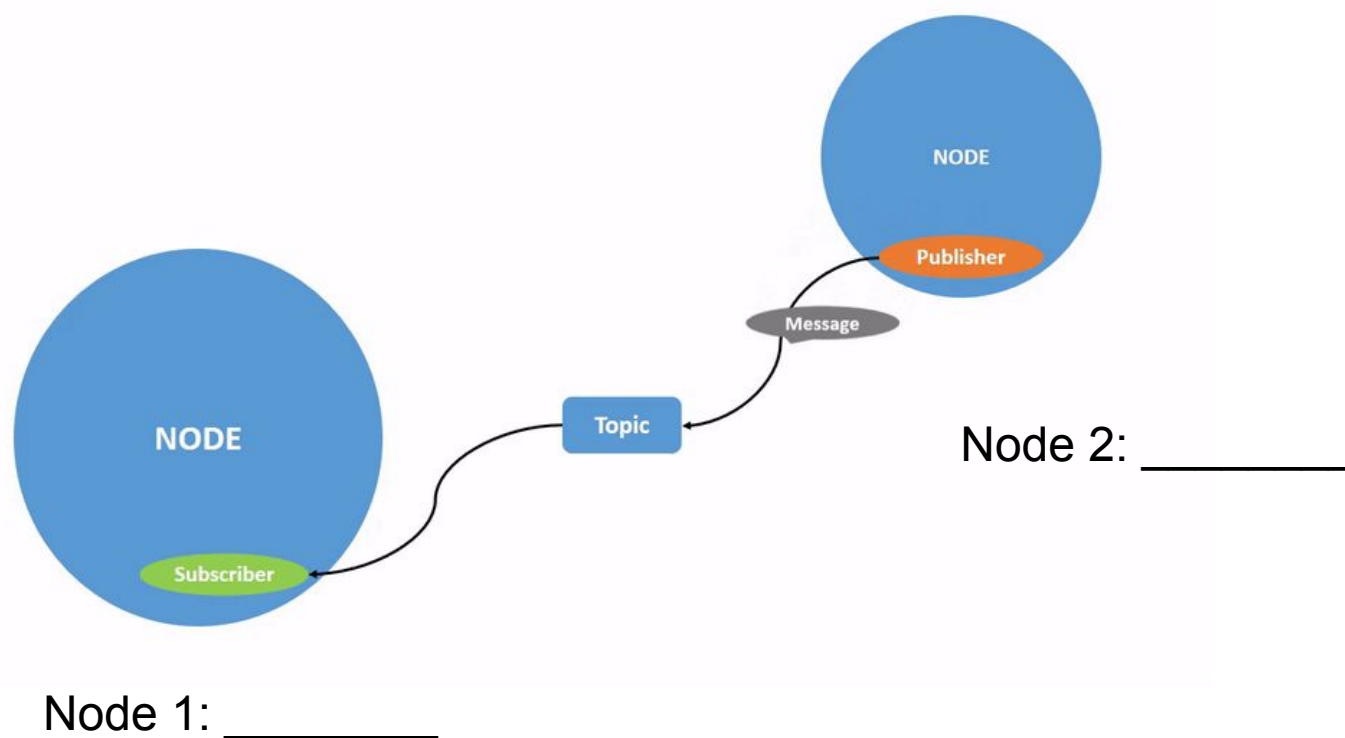


Turtlesim



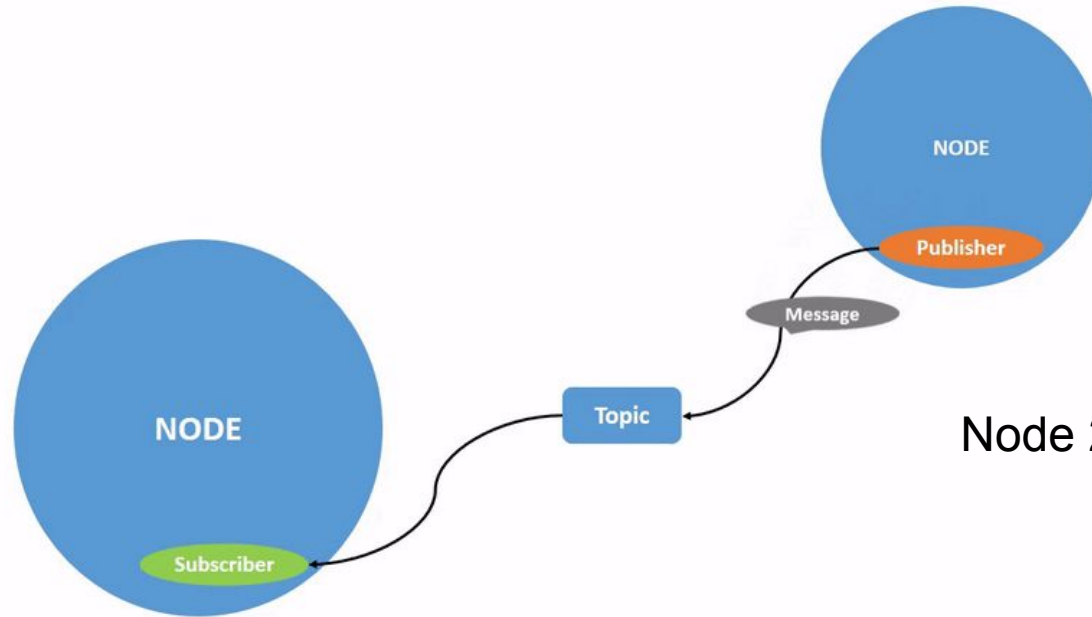
Nodes in ROS2

Guess the node name.



```
ros2 run turtlesim turtlesim_node
```


Nodes in ROS2



Node 1: turtlesim_node

`ros2 run turtlesim turtlesim_node`

Node 2: turtle_teleop_key

`ros2 run turtlesim turtle_teleop_key`

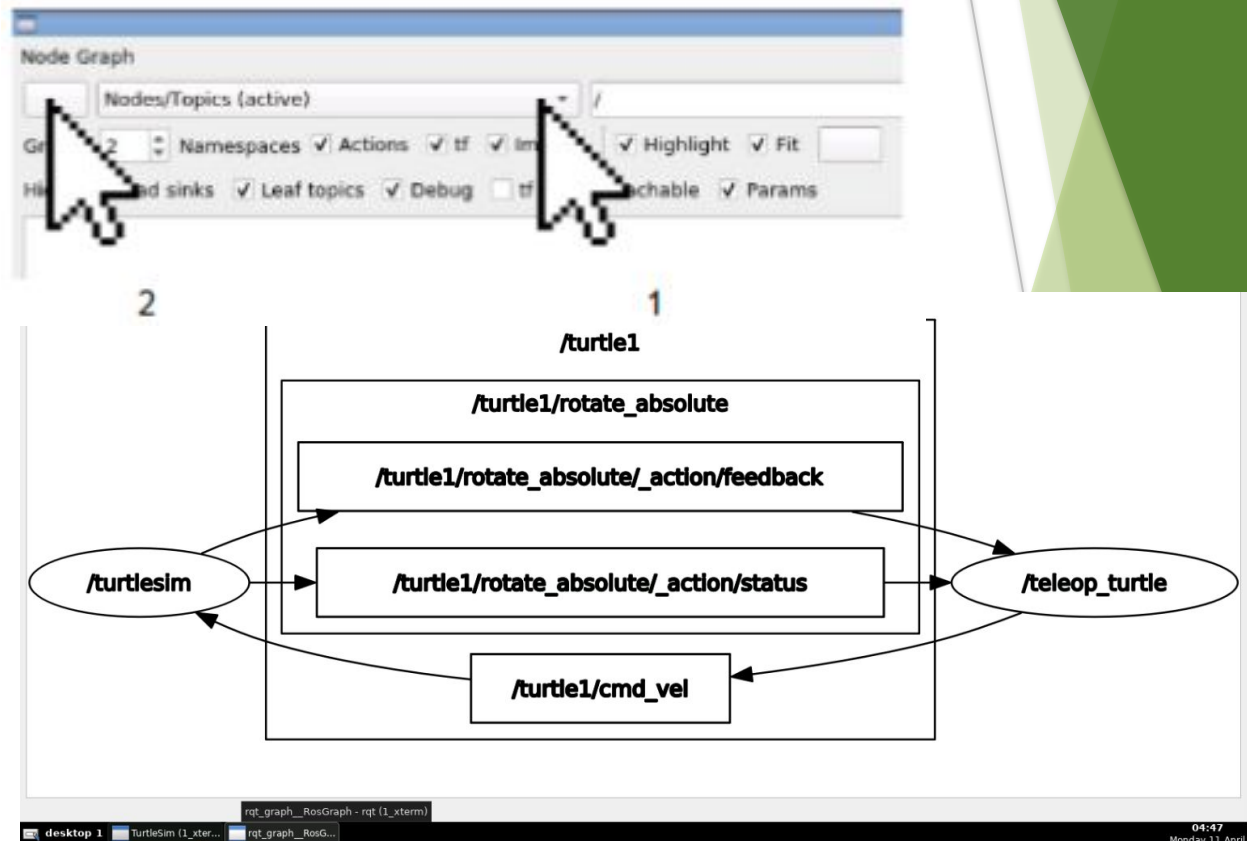
Nodes in ROS2

Open a 4th terminal and enter the following command:

```
$ rqt_graph
```

rqt_graph is used to visualize the changing nodes and topics, as well as the connections between them. Make sure to click the refresh button for any changes.

The graph is depicting how the /turtlesim node and the /teleop_turtle node are communicating with each other over a topic. The /teleop_turtle node is publishing data (the keystrokes you enter to move the turtle around) to the /turtle1/cmd_vel topic, and the /turtlesim node is subscribed to that topic to receive the data. The highlighting feature of rqt_graph is very helpful when examining more complex systems with many nodes and topics connected in many different ways. rqt_graph is a graphical introspection tool. Now we'll look at some command line tools for introspecting topics.



Ros2 topic echo

Open a 5th terminal and enter the following command:

```
$ ros2 topic echo /turtle1/cmd_vel
```

Return to the terminal where turtle_teleop_key is running and use the arrows to move the turtle around. Watch the terminal where your echo is running at the same time, and you'll see position data being published for every movement you make

linear:

x: 2.0

y: 0.0

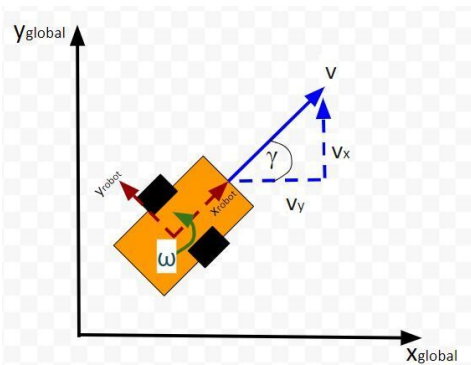
z: 0.0

angular:

x: 0.0

y: 0.0

z: 0.0



ROS 2 commands

Open a new shell/terminal for each task below and observe the output

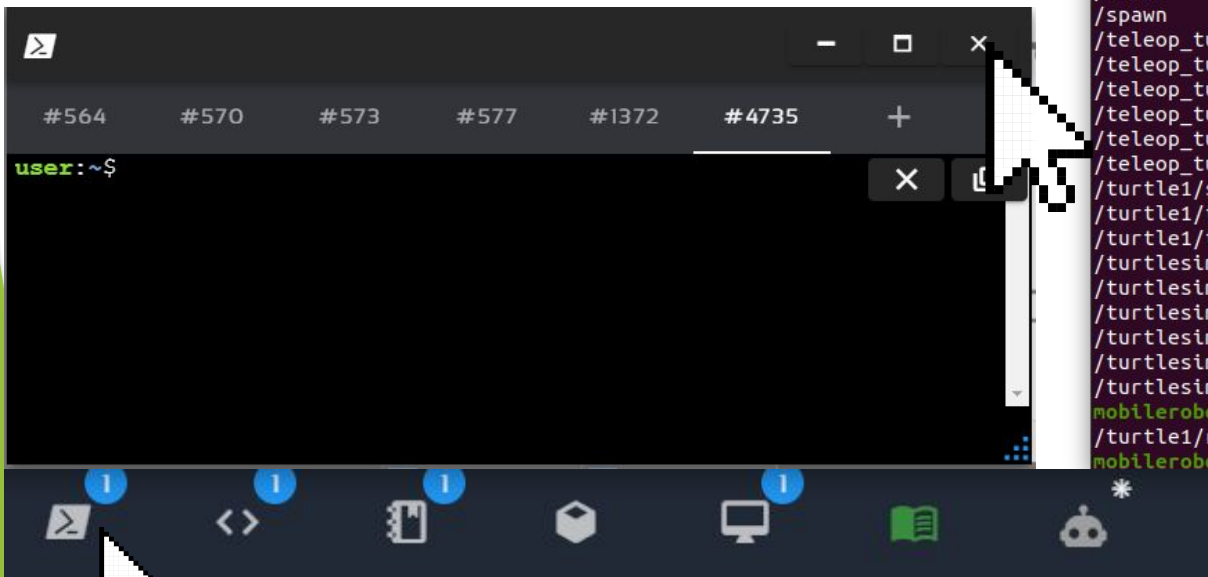
ros2 node list

ros2 topic list

ros2 service list

ros2 action list

You can close all terminal once you are done.



```
mobilerobot@ubuntu: ~  
mobilerobot@ubuntu:~$ ros2 node list  
/teleop_turtle  
/turtlesim  
mobilerobot@ubuntu:~$ ros2 topics list  
usage: ros2 [-h]  
          Call 'ros2 <command> -h' for more detailed  
          usage. ...  
ros2: error: argument Call 'ros2 <command> -h' for more de  
tailed usage.: invalid choice: 'topics' (choose from 'acti  
on', 'bag', 'component', 'control', 'daemon', 'doctor', 'e  
xtension_points', 'extensions', 'interface', 'launch', 'li  
fecycle', 'multicast', 'node', 'param', 'pkg', 'run', 'sec  
urity', 'service', 'topic', 'wtf')  
mobilerobot@ubuntu:~$ ros2 topic list  
/parameter_events  
/rosout  
/turtle1/cmd_vel  
/turtle1/color_sensor  
/turtle1/pose  
mobilerobot@ubuntu:~$ ros2 service list  
/clear  
/kill  
/reset  
/spawn  
/teleop_turtle/describe_parameters  
/teleop_turtle/get_parameter_types  
/teleop_turtle/get_parameters  
/teleop_turtle/list_parameters  
/teleop_turtle/set_parameters  
/teleop_turtle/set_parameters_atomically  
/turtle1/set_pen  
/turtle1/teleport_absolute  
/turtle1/teleport_relative  
/turtlesim/describe_parameters  
/turtlesim/get_parameter_types  
/turtlesim/get_parameters  
/turtlesim/list_parameters  
/turtlesim/set_parameters  
/turtlesim/set_parameters_atomically  
mobilerobot@ubuntu:~$ ros2 action list  
/turtle1/rotate_absolute  
mobilerobot@ubuntu:~$
```

Ros2 topic pub

Close all existing terminal and shell using (Ctrl +C on each terminal)

Open a new shell as shown



Terminal 1: `ros2 run turtlesim turtlesim_node`

Terminal 2: `ros2 run turtlesim turtle_teleop_key`

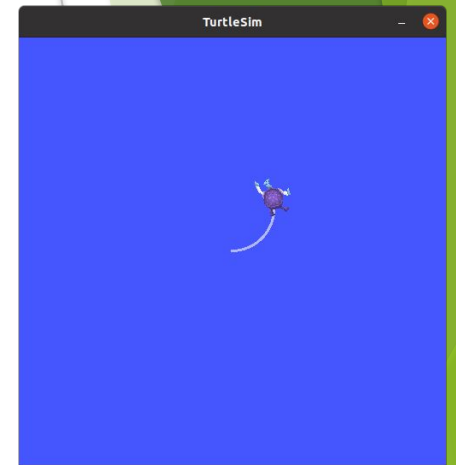
Terminal 3: `ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"`

Now that you have the message structure, you can publish data onto a topic directly from the command line using:

`ros2 topic pub <topic_name> <msg_type> '<args>'`

Linear Speed = (Distance / time) = d / t , The unit of linear speed is m/s or ms^{-1} .

The unit of Angular speed is radian per second (rad/s).



```
$ ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

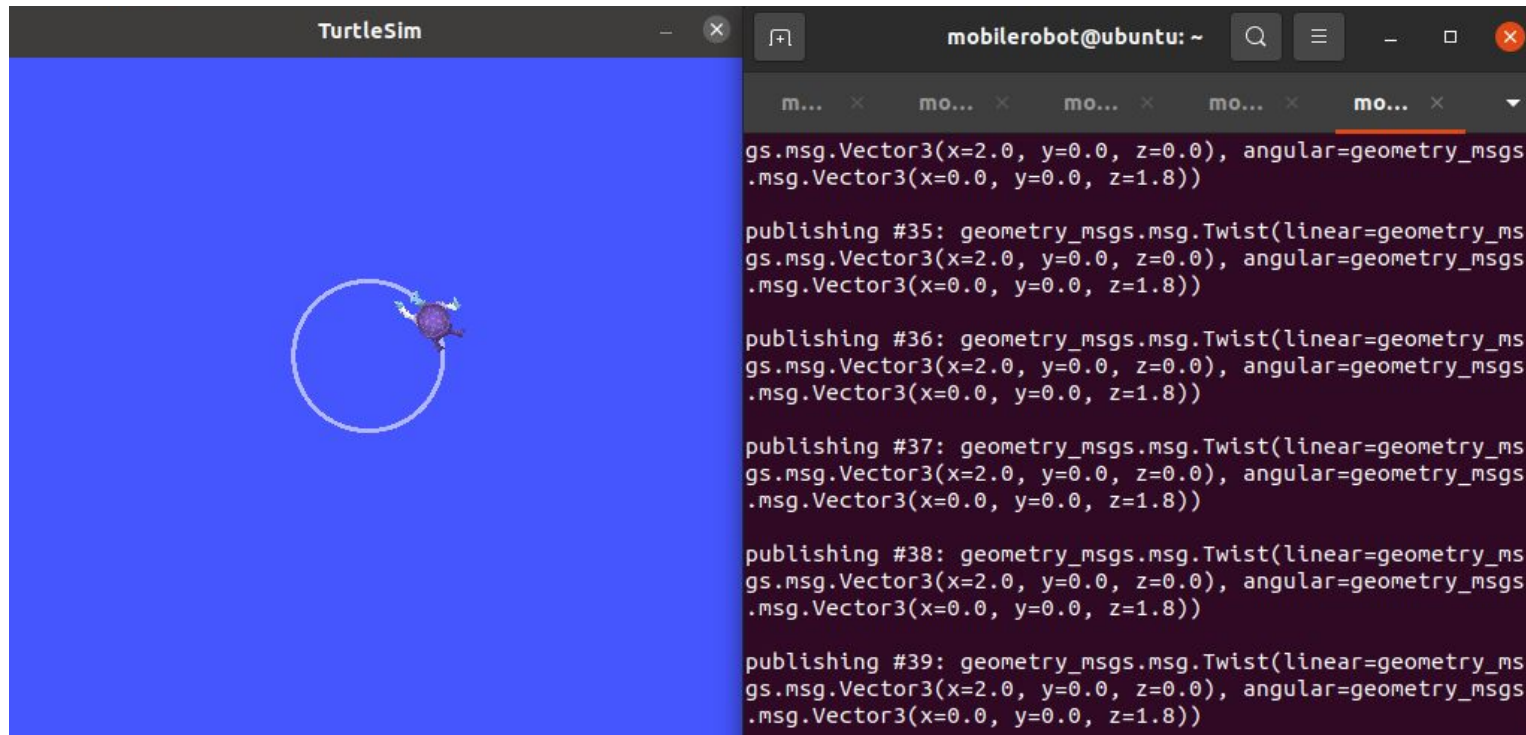
publisher: beginning loop

publishing #1: `geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=1.8))`

Ros2 topic pub

Try the following and note the difference in the command and the output. Explain.
You can stop the publisher using Ctrl+C

```
$ ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

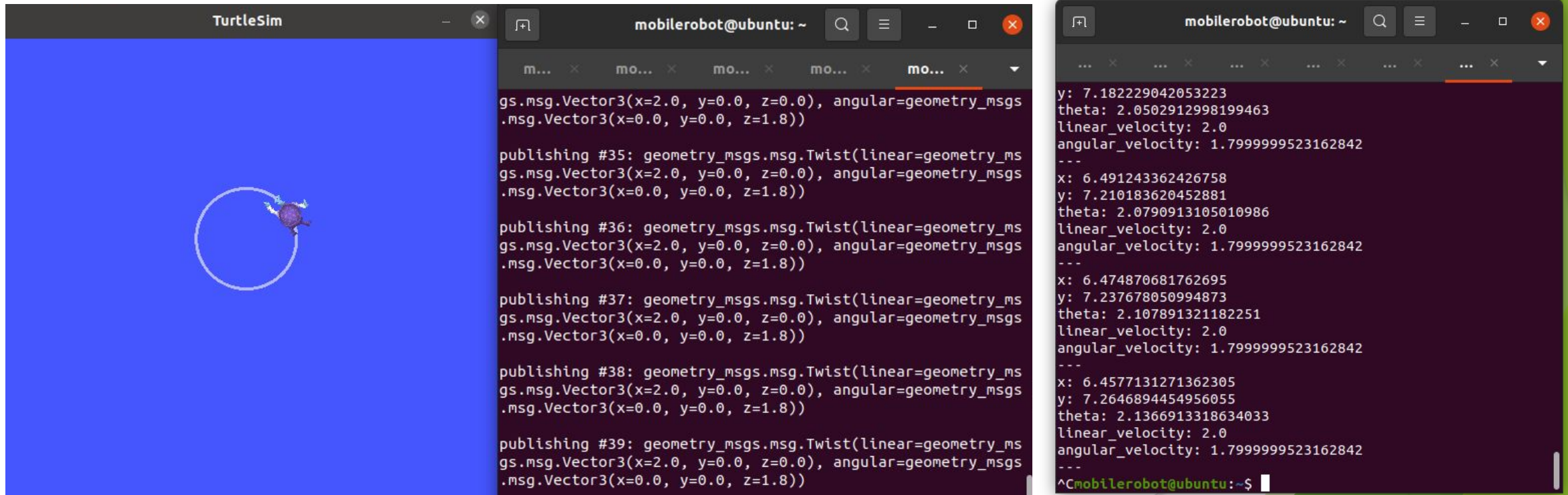


Ros2 topic echo

Try the following and observe the the linear_velocity and angular velocity.

```
$ ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

```
$ ros2 topic echo /turtle1/pose
```



The image shows a TurtleSim window on the left, displaying a purple turtle moving in a circle on a blue background. To the right are two terminal windows. The first terminal window, titled 'mobilerobot@ubuntu: ~', shows the output of the 'ros2 topic pub' command, displaying the Twist message being published. The second terminal window, also titled 'mobilerobot@ubuntu: ~', shows the output of the 'ros2 topic echo /turtle1/pose' command, displaying the turtle's pose data, including position (x, y), orientation (theta), and velocity (linear_velocity, angular_velocity).

```
gs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs
.msg.Vector3(x=0.0, y=0.0, z=1.8))

publishing #35: geometry_msgs.msg.Twist(linear=geometry_ms
gs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs
.msg.Vector3(x=0.0, y=0.0, z=1.8))

publishing #36: geometry_msgs.msg.Twist(linear=geometry_ms
gs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs
.msg.Vector3(x=0.0, y=0.0, z=1.8))

publishing #37: geometry_msgs.msg.Twist(linear=geometry_ms
gs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs
.msg.Vector3(x=0.0, y=0.0, z=1.8))

publishing #38: geometry_msgs.msg.Twist(linear=geometry_ms
gs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs
.msg.Vector3(x=0.0, y=0.0, z=1.8))

publishing #39: geometry_msgs.msg.Twist(linear=geometry_ms
gs.msg.Vector3(x=2.0, y=0.0, z=0.0), angular=geometry_msgs
.msg.Vector3(x=0.0, y=0.0, z=1.8))

y: 7.182229042053223
theta: 2.0502912998199463
linear_velocity: 2.0
angular_velocity: 1.7999999523162842
---
x: 6.491243362426758
y: 7.210183620452881
theta: 2.0790913105010986
linear_velocity: 2.0
angular_velocity: 1.7999999523162842
---
x: 6.474870681762695
y: 7.237678050994873
theta: 2.107891321182251
linear_velocity: 2.0
angular_velocity: 1.7999999523162842
---
x: 6.4577131271362305
y: 7.2646894454956055
theta: 2.1366913318634033
linear_velocity: 2.0
angular_velocity: 1.7999999523162842
---
^Cmobilerobot@ubuntu:~$
```

Change the publisher linear x to 4.0 and angular z to 5.8 and observe the changes in the topic echo

Change the publisher linear x to 1.0 and angular z to 0.8 and observe the changes in the topic echo

Ros2 topic hz

Run the following command to publish the command velocity

```
$ ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

Open a new terminal and run the following to view the rate of topic being published

```
$ ros2 topic hz /turtle1/cmd_vel
```

```
average rate: 1.000  
  min: 0.998s max: 1.001s std dev: 0.00065s window: 22
```

Modify the command such that the average rate is 2.0 (2 times per second)

Press Ctrl+C to stop the process for each terminal. After stopping all the process, you can close the terminal and shut down the PC,

Be prepared to answer these question during demo.

Q1) Is turtle_teleop_key a subscriber or publisher ? Explain.

Q2) What is the topic that turtlesim node subscribe to ?

Q3) Explain the function of this command line "ros2 topic pub <topic_name> <msg_type> '<args>' "

Q4) Explain the difference between linear and angular velocity

End of briefing for Lab session

Be prepared to answer these question during demo.

Q1) Is turtle_teleop_key a subscriber or publisher ? Explain.

Q2) what is the topic that turtlesim node subscribe to ?

Q3) Explain the function of this command line "ros2 topic pub <topic_name> <msg_type> '<args>' "

Q4) Explain the difference between linear and angular velocity

Q1) Publisher, from the rqt_graph, it publish a topic, it does not subscribe to any topic

Q2) turtle1/cmd_vel

Q3) \$ ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"

it publish a topic with msg type and arguments.

Q4) Linear and angular are two different types of motion that objects can undergo.

Linear motion is motion in a straight line. On the other hand, angular motion is defined as the motion of a body about a fixed point or fixed axis

The key difference between linear and angular motion lies in the path that the moving object follows. In linear motion, the object moves along a straight path, whereas in angular motion, the object moves along a curved path